Smart Home Automation Hub

Assignment on Structural Design Patterns

CSE 213 — Software Engineering

Problem Description

A smart home company, NexaHome, is developing a system to control an entire smart home along with its individual smart devices. Each room of a smart home can have any number of smart devices installed. For now, NexaHome wants to support three device types:

- **SmartLight —** can be turned on/off; consumes 10W when active.
- **SmartThermostat —** can be turned on/off; consumes 150W when active.
- **SmartSpeaker —** can play or stop audio; consumes 5W when active.

A room might have, say, 2 speakers and 4 lights, while another room might have 1 thermostat and 1 light — there is no fixed layout. Devices are installed over time, so the system must allow adding and removing devices from any room at any point.

Every entity in the system, be it a single device, a room containing a collection of devices, or the entire home, must support the same set of operations: querying status, reporting power usage in watts (0 when inactive), and activating or deactivating. When a user activates a room, all devices in that room should activate; when they activate the home, every device in every room should respond through the same uniform command, with no separate handling for different levels.

Device Upgrade

NexaHome offers upgrade options for individual devices. A client can upgrade any device by paying an extra charge, unlocking additional capabilities for that specific device. The available upgrade options are:

AccessRestricted

Makes a device PIN-protected. A locked device silently ignores all activate/deactivate commands until unlocked with the correct PIN. Its status should indicate the locked state. However, a locked device still draws power if it was already running before being locked — locking does not cut the power, it only blocks further control.

TimerControlled

Adds an automatic shutoff timer. When the device is activated, a countdown begins. When the timer expires, the device is automatically deactivated. The status should show the remaining time.

> *Note: NexaHome does not currently support TimerControlled for SmartSpeaker due to audio streaming constraints, but this may change soon. Nothing in your code should prevent it from*

> *compiling if someone applies it to a speaker. This is a policy constraint, not a type-system constraint.*

PowerThrottled

Limits a device's power draw to a specified cap. For example, a thermostat normally drawing 150W can be throttled to 80W, forcing it to operate at reduced capacity. The device's reported power usage reflects this real reduction. This is useful for devices installed on circuits with limited capacity or in rooms where energy conservation is a priority.

Critically: a single device can have multiple upgrades active simultaneously. For instance, a SmartThermostat might be both AccessRestricted and PowerThrottled simultaneously. A room can contain a mix of plain devices and upgraded devices side by side, and the system must treat them all uniformly — no special-casing for upgraded vs. standard devices in the room's logic.

Room-Level and Home-Level Features

If a client upgrades 5 or more devices (i.e., pays for 5+ device-level upgrades), they unlock NexaHome's Premium Plan, which offers features that operate at the room level or even the entire home level:

Eco Mode

Enforces a total power budget (in watts) on a room or the whole home. When activated:

- If the total power consumption stays within the budget, everything works normally.
- If the total exceeds the budget, devices are automatically deactivated in reverse order of installation (most recently added devices are shed first) until the total fits within the budget.
- The status should clearly indicate the active budget.

Important: EcoMode is a constraint on the aggregate total, not on individual devices. A room with three 60W devices under a 100W budget should keep one or two devices active — it should NOT throttle each device down to 33W. EcoMode and PowerThrottled are fundamentally different operations even though both relate to power.

Guest Mode

Puts a room or the entire home in guest mode, restricting which types of devices guests are allowed to operate. The client specifies a set of allowed device types (e.g., lights and speakers, but not thermostats). When GuestMode is active:

- Only allowed device types respond to activation; others are silently skipped.
- The status marks non-allowed devices as guest-restricted.
- Power usage reports only the consumption of allowed devices.

A room can be put in EcoMode, GuestMode, or both simultaneously, and the system must handle both constraints together.

Your Task

An amateur developer wrote code that works but is brittle i.e., every small change introduces bugs across the codebase. Frustrated by this, NexaHome has hired you to redesign and reimplement the system properly.

You are given the existing spaghetti implementation in a single SmartHome.java file, along with a SmartHomeTestRunner.java test suite. Your task is to refactor SmartHome.java so that the design satisfies the SOLID principles while supporting all of the following:

1. Uniform handling. Every entity, a single device, a room, the entire home, or any upgraded version of any of these, must be usable through the same contract. Code that operates on a single SmartLight must also work, without modification, on an entire room or an upgraded device.

2. Composability. Enhancements must be freely combinable. A PowerThrottled thermostat inside an EcoMode room must work. An AccessRestricted room (yes, the entire room — locked until a PIN is entered) must work. Your architecture must not artificially restrict which enhancements can be applied to which entities.

3. No special-casing. A room's logic for cascading activate/deactivate and summing power usage must NOT contain "if" checks for whether a child is plain, upgraded, or premium. The uniform interface must make this unnecessary.

4. Order sensitivity. Applying PowerThrottled to a thermostat before adding it to an EcoMode room should produce a different result than applying EcoMode to a room with a raw thermostat. Your demo must demonstrate this concretely.

Submission

You are given SmartHomeSpaghetti.java (the spaghetti implementation) and SmartHomeTestRunner.java (the test suite). Refactor SmartHome.java only. Do not modify SmartHomeTestRunner.java. After refactoring, all tests must pass. You have to submit your refactored Smarthome.java along with the test file SmartHomeTestRunner.java, zipped and renamed to your student ID.